NDFEX ETF Service

REST API Documentation

The ETF Service provides a REST API for creating and redeeming shares of the **UNDY** (Notre Dame Dorm ETF). One UNDY share represents one share of each of the 10 underlying dorm symbols.

REST API	`http://129.74.160.245:5000`
WebSocket	`ws://129.74.160.245:9003`
Authentication	`HTTP Basic Auth (team credentials)`

ETF Composition

1 UNDY = 1 share of each underlying dorm symbol:

Symbol ID	Ticker	Name	Tick Size
3	KNAN	Keenan Hall	5
4	STED	St. Edward's Hall	5
5	FISH	Fisher Hall	5
6	DILN	Dillon Hall	5
7	SORN	Sorin Hall	5
8	RYAN	Ryan Hall	5
9	LYON	Lyons Hall	5
10	WLSH	Walsh Hall	5
11	LEWI	Lewis Hall	5
12	BDIN	Badin Hall	5
13	UNDY	Notre Dame Dorm ETF	10

Additional trading symbols: GOLD (ID 1, tick 10) and BLUE (ID 2, tick 5) are not part of the ETF basket.

Authentication

State-mutating endpoints (**/create**, **/redeem**) and the **/whoami** endpoint require HTTP Basic Auth. Credentials are the same team name and password issued for the matching engine (distributed to your team — see your team lead). The authenticated user is bound to one client_id; you may only create or redeem on your own behalf.

Read-only endpoints (**/health**, **/symbols**, **/positions/<id>**, **/history**) remain public so the dashboard and other observers can monitor the market without credentials.

How to authenticate:

• **curl:** pass -u <team_name>:<password> on every request.

• **Python requests:** pass auth=(team_name, password).

• **Browser (web UI):** the page prompts for credentials on first load; the browser remembers them for the session.

Auth error responses:

Status	Cause
`401 Unauthorized`	Missing or invalid credentials
`403 Forbidden`	Body client_id does not match authenticated user

API Endpoints

Health Check

`GET /health`

Returns service health status. Use for monitoring. **No auth required.**

```
{
  "status": "ok",
  "service": "etf_service"
}
```

List Symbols

`GET /symbols`

Returns all symbol definitions, the ETF symbol ID, and the list of underlying symbol IDs.

```
{
  "symbols": [
    {"id": 1, "ticker": "GOLD", "name": "Gold", "tick_size": 10},
    {"id": 2, "ticker": "BLUE", "name": "Blue", "tick_size": 5},
    ...
  ],
  "etf_symbol": 13,
  "underlying_symbols": [3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
}
```

Get Client Positions

`GET /positions/<client_id>`

Returns all positions for the given client. Positions combine clearing fills with ETF create/redeem adjustments. Only non-zero positions are included.

```
GET /positions/1

{
  "client_id": 1,
  "positions": {
    "GOLD": 50,
    "KNAN": 10,
    "UNDY": 5
  }
}
```

Get Single Position

`GET /positions/<client_id>/<symbol>`

Returns the position for a specific client and symbol ID.

```
GET /positions/1/13

{
  "client_id": 1,
  "symbol": 13,
  "ticker": "UNDY",
  "position": 5
}
```

Identify Authenticated User

`GET /whoami`

Requires auth. Returns the client_id and team name bound to the supplied credentials. Useful for sanity-checking your login.

```
GET /whoami

{
  "client_id": 1,
  "name": "team1"
}
```

Create ETF Shares

`POST /create`

Requires auth. Exchange underlying dorm positions for UNDY ETF shares for the authenticated client. The client must hold at least **amount** shares of *each* of the 10 underlying symbols. The operation is atomic: all underlying positions are debited and UNDY is credited in one step.

Request: the client_id is taken from your authenticated credentials. You may optionally include client_id in the body, but it must match — otherwise the request is rejected with 403.

```
POST /create
Authorization: Basic <base64(team_name:password)>
Content-Type: application/json

{
  "amount": 10
}
```

Success Response (200):

```
{
  "success": true,
  "message": "Created 10 UNDY from underlying positions",
  "undy_balance": 15
}
```

Error Response (400):

```
{
  "success": false,
  "message": "Insufficient positions: KNAN: have 5, need 10",
  "undy_balance": 5
}
```

Redeem ETF Shares

`POST /redeem`

Requires auth. Exchange UNDY ETF shares back into underlying dorm positions for the authenticated client. The client must hold at least **amount** UNDY shares. Each underlying symbol is credited with **amount** shares.

Request:

```
POST /redeem
Authorization: Basic <base64(team_name:password)>
Content-Type: application/json

{
  "amount": 5
}
```

Success Response (200):

```
{
  "success": true,
  "message": "Redeemed 5 UNDY to underlying positions",
  "undy_balance": 10
}
```

Error Response (400):

```
{
  "success": false,
  "message": "Insufficient UNDY: have 5, need 10",
  "undy_balance": 5
```

```
}
```

Transaction History

`GET /history`

Returns the full create/redeem history for auditing.

```
{
  "history": [
    {"type": "create", "client_id": 1, "amount": 10},
    {"type": "redeem", "client_id": 1, "amount": 5}
  ]
}
```

Error Reference

Error responses use this JSON shape (the **undy_balance** field is omitted on auth failures):

```
{
  "success": false,
  "message": "<error details>",
  "undy_balance": <current UNDY balance>
}
```

Status / Error	Cause
`401 Authentication required`	No Authorization header on a protected endpoint
`401 Invalid credentials`	Wrong team name or password
`403 client_id does not match`	Body client_id is not your authenticated client_id
`400 Missing amount`	amount field absent from JSON body
`400 Invalid amount`	amount not convertible to integer
`400 Invalid client_id`	client_id field present but not an integer
`400 Amount must be positive`	amount <= 0
`400 Insufficient positions: ...`	Not enough underlying shares to create
`400 Insufficient UNDY: ...`	Not enough UNDY shares to redeem

Usage Examples (curl)

Substitute your team name and password (issued separately) for team1:PASSWORD in the examples below.

Check service health (no auth):

```
curl http://129.74.160.245:5000/health
```

Get positions for client 1 (no auth):

```
curl http://129.74.160.245:5000/positions/1
```

Confirm your login:

```
curl -u team1:PASSWORD http://129.74.160.245:5000/whoami
```

Create 10 UNDY shares (auth required):

```
curl -u team1:PASSWORD -X POST http://129.74.160.245:5000/create \
  -H "Content-Type: application/json" \
  -d '{"amount": 10}'
```

Redeem 5 UNDY shares (auth required):

```
curl -u team1:PASSWORD -X POST http://129.74.160.245:5000/redeem \
  -H "Content-Type: application/json" \
  -d '{"amount": 5}'
```

Python example:

```
import requests

BASE = "http://129.74.160.245:5000"
AUTH = ("team1", "PASSWORD")  # your team credentials

# Check positions (no auth needed)
r = requests.get(f"{BASE}/positions/1")
print(r.json())

# Confirm login
r = requests.get(f"{BASE}/whoami", auth=AUTH)
print(r.json())  # {"client_id": 1, "name": "team1"}

# Create ETF shares for the authenticated client
r = requests.post(f"{BASE}/create", auth=AUTH,
    json={"amount": 10})
print(r.json())
```

Position Calculation

The effective position for any client/symbol pair is:

```
effective_position = clearing_position + etf_adjustment
```

clearing_position comes from fill messages on the clearing multicast feed (trades executed on the matching engine).

etf_adjustment comes from create/redeem operations through this API. Creating UNDY debits underlyings and credits UNDY; redeeming does the reverse.

> *The ETF service maintains its own position ledger. Create/redeem adjustments are tracked locally and do not generate fills on the matching engine.*